



COMPETITIVE PROGRAMMING ROADMAP



Step-by-Step Guide to Become a Better Problem Solver & CP Expert

LEARN → PRACTICE → SOLVE → MASTER → SUCCEED

1		1. Programming Fundamentals - Choose a language: C++ (recommended) or Python/Java - Basics: Variables, Loops, Functions, I/O - Practice basic problems	Duration ⌚ 1–2 Weeks Resources → Codeforces – Practice → HackerRank – Tutorial
2		2. Basic Data Structures - Arrays, Strings, Vectors - Stack, Queue, Deque - Set, Map, Pair	Duration ⌚ 2–3 Weeks Resources → CP-Algorithms → LeetCode Explore
3		3. Advanced Data Structures - Linked List (Singly, Doubly) - Priority Queue / Heap - Hashing (Maps, Sets, Hash Function)	Duration ⌚ 2–3 Weeks Resources → CP-Algorithms → GeeksforGeeks
4		4. Trees - Binary Tree Basics - Binary Search Tree - Tree Traversals (Inorder, Preorder, Postorder) - Lowest Common Ancestor, Tree DP	Duration ⌚ 2–3 Weeks Resources → CP-Algorithms → LeetCode / GFG
5		5. Graphs - Graph Representations (Adj List, Matrix) - BFS, DFS - Shortest Path (Dijkstra, Bellman-Ford) - Topological Sort, DSU	Duration ⌚ 3–4 Weeks Resources → CP-Algorithms → LeetCode / GFG
6		6. Mathematics for CP - Number Theory (GCD, LCM, Primes) - Modular Arithmetic - Combinatorics (nCr, Permutations) - Mathematics Tricks	Duration ⌚ 2–3 Weeks Resources → CP-Algorithms → Math Blog, Errichto
7		7. Algorithms - Sorting (Merge, Quick, Counting) - Binary Search & Variations - Two Pointers, Sliding Window - Greedy Algorithms	Duration ⌚ 2–3 Weeks Resources → CP-Algorithms → LeetCode / GFG
8		8. Dynamic Programming 1D DP, 2D DP - Knapsack Variants - DP on Trees, Bitmask DP	Duration ⌚ 3–4 Weeks Resources → CP-Algorithms → YDP (Practice)
9		9. Advanced Topics - Backtracking - Divide & Conquer - Binary Lifting - Segment Tree, Fenwick Tree (BIT) - Trie	Duration ⌚ 3–4 Weeks Resources → CP-Algorithms → GeeksforGeeks
10		10. Problem Solving Practice - Start solving easy problems regularly - Focus on patterns & techniques - Participate in contests	Duration ⌚ Ongoing Resources → AtCoder, LeetCode
11		11. Contest & Advanced Practice - Participate in regular contests - Analyze editorial & learn new ideas - Track progress & improve rating	Duration ⌚ Ongoing Resources → AtCoder, CodeChef
12		12. Refinement & Mastery - Solve harder problems - Learn advanced techniques - Build speed + accuracy	Duration ⌚ Ongoing → Virtual Contests → ICPC Archive, Kattis

KEY SKILLS YOU WILL BUILD

-  Problem Solving
-  Analytical Thinking
-  Algorithm Design
-  Time & Space Optimization
-  Implementation Skills
-  Logical Thinking
-  Debugging & Testing

BEST PLATFORMS TO PRACTICE

-  **Codeforces**
(Contests & Problems)
-  **AtCoder**
(High Quality Contests)
-  **LeetCode**
(Practice & Patterns)
-  **CodeChef**
(Contests & Learning)
-  **HackerRank**
(Practice & Basic CP)
-  **CS Academy**
(Round Based Contests)
-  **Kattis**
(Practice & Contests)

STUDY PLAN (SUGGESTED)

-  Learn 2–3 hours daily
-  Solve at least 2–3 problems daily
-  1–2 contests every week
-  Read editorials and upsolve
-  Revise concepts regularly
-  Track progress & stay consistent

TIPS FOR SUCCESS

- ★ Focus on understanding, not memorizing
- ★ Solve by yourself before looking at solution
- ★ Learn from editorials
- ★ Keep a notebook of tricks & templates
- ★ Don't get stuck for too long on one problem
- ★ Enjoy the process and stay consistent

CAREER OPPORTUNITIES

-  Software Engineer (SDE)
-  Competitive Programmer
-  Backend Developer
-  Full Stack Developer
-  Data Engineer / Analyst
- AI / ML Engineer



CONSISTENCY + PRACTICE + ANALYSIS = CP SUCCESS

-  Solve Daily
-  Think Deep
-  Learn New
-  Never Give Up